

EscapeArrow project manual

Player guide, administrator guide and implementation reference

Version 1.0 • Source release • 19 September 2026

EscapeArrow is an online arrow puzzle with an Android player app, a browser player interface and a separate administrator dashboard. This manual explains how to play, operate the dashboard, run the code, connect PostgreSQL, configure account providers and build an Android APK. The supplied ZIP contains editable modules, the generated Android project, API definitions and tests.

What the release contains

Part	Purpose	Entry point
Player app	Play puzzles, earn rewards, edit profile, review the game	client/index.html
Administrator dashboard	Inspect players, attempts, reviews and aggregate metrics	admin/index.html
Backend	Verify accounts and control game state and rewards	backend/app/main.py
Android project	Package the player interface as an Android application	android/
Documentation	Manual, API schema, database schema and diagrams	docs/

Build status

The API tests and frontend build have run successfully, and the Android project has been generated and synced. This source release does not contain a compiled or signed APK. Real email/SMS, Google OAuth, production PostgreSQL and physical-device behavior need verification after configuration. No provider credentials or hosted accounts are supplied.

How to read this manual

Sections	Read this when
2 to 4	You want to understand accounts, gameplay, ratings and rewards.
5	You want to use the administrator dashboard.
6 to 8	You want to understand modules, data and authentication.
9 to 12	You want to run locally, deploy, connect providers and build an APK.
13 to 15	You want the API reference, tests and troubleshooting.

2 Player accounts and profiles

Create an account

1. Open the player app and select Sign up. Enter your name, a game username, an email address or international phone number, and a password. A phone number includes its country code, for example +91 followed by the number.
2. Use 3 to 24 letters, digits or underscores for the username. Passwords must be at least eight characters and no more than 72 UTF-8 bytes.
3. Select Create account. The server generates a four-digit code and sends it to the selected contact. In local development, read the code from the backend terminal instead.
4. Enter the code within five minutes. Five incorrect attempts exhaust the challenge. Requesting a new code invalidates the previous one.
5. After successful verification, the server assigns a random 12-digit player ID. A database uniqueness constraint prevents duplicate IDs. The profile receives the registration name and verified contact.

Sign in and recover your account

Sign in accepts the registered email or phone plus password, followed by an OTP. Use OTP signs in through the verified contact without a password. Forgot password sends a code to the registered contact, then accepts a new password. A successful reset revokes all older sessions. An unknown contact returns No user found as requested.

Google sign in

Continue with Google opens the system browser on Android. Finish Google sign-in, return to the game and select I have signed in. Enter the additional four-digit code sent to your Google email. Google provides the name and verified email. A first-time Google account receives a temporary game username; change it in Profile. Set a password through Forgot password if you want password sign-in too.

Maintain your profile

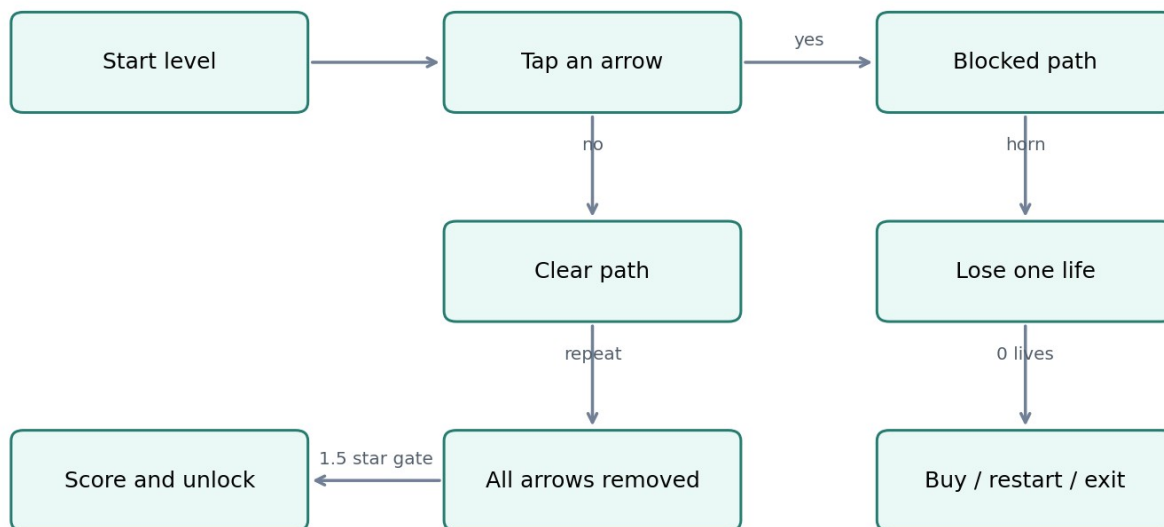
Select Profile to change your username and gender. Choose a picture from the gallery picker; accepted image bytes are decoded, resized to at most 256 by 256 pixels and stored as JPEG. The upload limit is 2 MB. Add or update a verified contact by entering an email or phone, requesting its OTP and verifying it. A contact already attached to an account cannot be reassigned through this flow.

The app cannot silently read your personal name, email or phone from the device. It stores the details you supply or authorize through Google. Unprovided fields show Not supplied. Your account details, attempts and reviews are visible to the authorized administrator.

3 Playing and completing levels

1. On Play, select the current unlocked level. Only level 1 is available for a new account. Older unlocked levels can be replayed.
2. Look along each arrow in the direction it points. Tap an arrow only when no other arrow lies between it and the grid edge. A clear arrow animates out of the board.
3. A blocked arrow flashes red, plays a short horn-like sound and costs one life. Each attempt begins with five lives. The timer runs from the server start time.
4. After all arrows leave, the server records elapsed seconds, collisions and a score from 0.5 to 5 stars. At least 1.5 stars unlocks the next level. A lower score requires another attempt.
5. Choose Collect rewards to receive the displayed coins and eligible diamonds. No thanks permanently declines the reward for that attempt. You can earn again by completing a fresh attempt.

Gameplay and recovery decisions



Lives and retries

At zero lives the board stops accepting moves. Choose Restart level for a fresh random board and five free lives, Exit to end the attempt, or buy one extra life with earned coins. Only three extra lives are available in one attempt, costing 10, 25 and 35 coins in that order. A restart resets the purchase counter. No real-money purchases are implemented.

Exit records an abandoned attempt. A disconnected player can resume the active attempt from home; elapsed time continues while away. Attempts older than 24 hours become abandoned when accessed. Only one active attempt per account is supported, so starting another level ends the earlier attempt.

4 Difficulty stars rewards and streaks

Difficulty	Levels	Coins per clear	Fast clear bonus
Easy	1 to 10	5	3 diamonds at 45 seconds or less
Moderate	11 to 30	7	3 diamonds at 45 seconds or less
Hard	31 onward	11	3 diamonds at 45 seconds or less

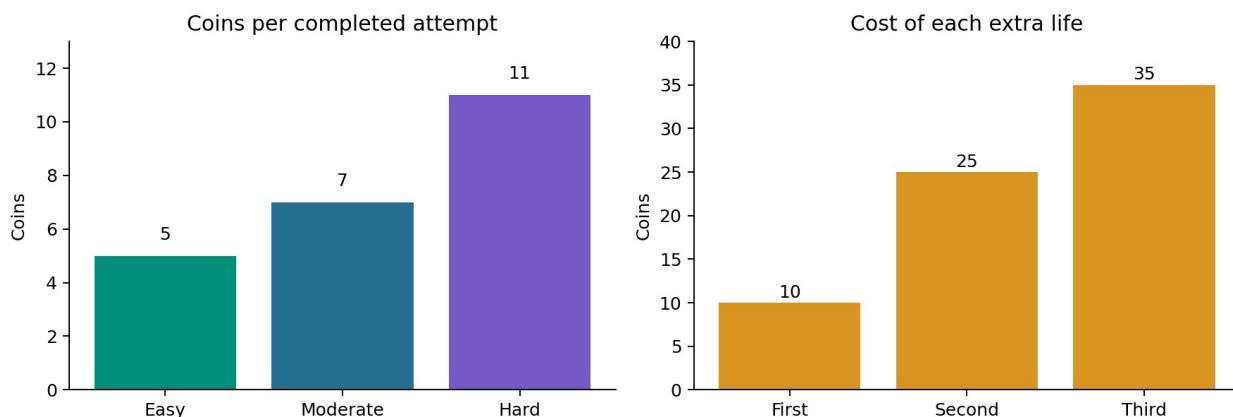
How difficulty increases

The board starts at 5 by 5 cells and grows by one row and column every ten levels, up to 10 by 10. Target arrow count is the smaller of grid cells minus three and 6 plus twice the level. Reverse construction places only arrows that can exit before older arrows, guaranteeing a solution. The actual count can be lower if no valid placements remain. Once board limits are reached, new randomized puzzles continue at the capped complexity; measured difficulty does not increase forever.

The star calculation

```
par = min(180, 45 + max(0, level - 10) * 2)
stars = round(clamp(5 - 0.65 * collisions
    - max(0, seconds - par) / 60, 0.5, 5), 1)
```

A clean run within par earns five stars. Each collision costs 0.65 before rounding. Each 60 seconds beyond par costs one star. For example, an easy level completed in 105 seconds with two collisions earns 2.7 stars. An easy level taking 300 seconds with two collisions reaches the 0.5-star floor and does not unlock the next level. Ratings use the recorded decimal value, not the decorative rounded star icons.



Daily play and seven day streaks

Starting a level qualifies as playing that day. The first qualifying day starts a one-day streak; consecutive dates extend it and a missed date resets it. Every seventh consecutive day automatically adds 50 coins and 20 diamonds, including day 14, day 21 and later multiples. The separate daily Claim button adds two coins once that day and plays the coin sound. Daily rewards must be claimed on the same date. The server calculates dates in the account time zone captured at registration; the displayed clock uses the device locale and time.

5 Administrator dashboard guide

Enable your administrator account

1. Register and verify your own account in the player app. There is no default administrator password.
2. Open a trusted terminal on the backend server. From backend, run `python manage.py make-admin your@email.com` using that server environment.
3. Open `https://YOUR-BACKEND-DOMAIN/admin/` and sign in with your verified account. The backend checks the admin flag for every reporting endpoint. Hiding a menu is not the permission control.

Read the overview

Metric or chart	Exact interpretation
Total players	Count of registered user rows, including administrator accounts.
Online now	Accounts whose heartbeat or game action occurred within the previous 90 seconds.
Levels cleared	Completed attempts, including replays. This is not unique levels or unique players.
Average clear time	Average server elapsed seconds of completed attempts; includes time away from the app.
Seven day bar chart	Number of attempts started on each of the last seven UTC dates.
Outcome donut	Completed attempts as a share of all attempts; other includes playing, exhausted and abandoned.
Difficulty comparison	Completions, average clear time and average star score by easy, moderate and hard.
Average game review	Average current review rating; each user has at most one current review.

Inspect a player

Open Players. Search by username, email or exact player ID. Browse in pages of 50. Select a username to see the name, verified contact details, gender, progression, coins, diamonds and the latest 200 attempts. Compare time, stars and collisions to understand performance. Longer attempt history remains in the database. The interface does not calculate foreground-only play time; use average clear time for the duration measure it actually records.

Read feedback

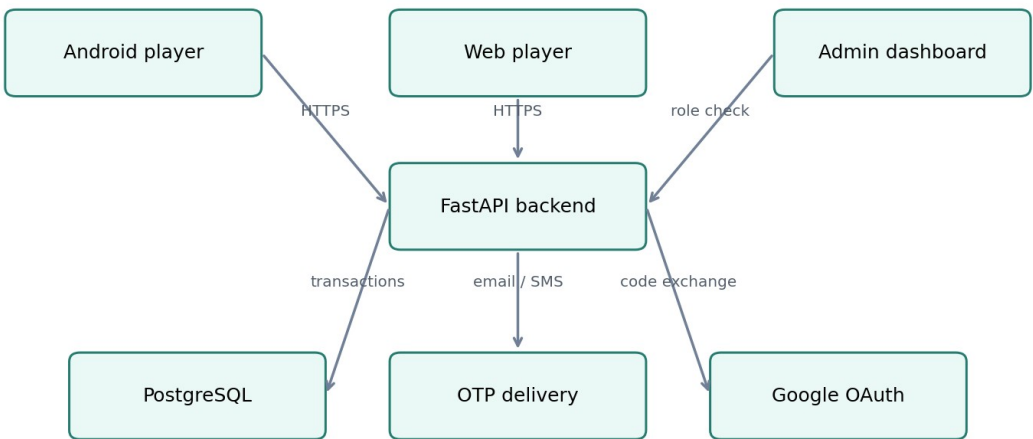
Open Reviews to see each player name, 1 to 5 stars, optional written suggestions and submission time. Review keywords in the player app are Poor, Fair, Good, Great and Excellent. Updating a review replaces that player's previous review. Overview refreshes every 30 seconds while visible. The dashboard uses real API data and displays empty states until players generate activity.

Access and operation

The admin interface is a separate browser dashboard; it is not bundled into the player APK. Database credentials, provider keys and role promotion stay on the server. Keep administrator access limited to the owner and explicitly authorized operators. Back up the database before deployments that change data structures.

6 Architecture and technology

One server connects both experiences



Technology	Where used	Why used
HTML CSS JavaScript	client and admin	Modular interface without a large UI framework; responsive grid and SVG arrows.
Web Audio API	client/src/audio.js	Generates collision, move and coin sounds locally.
CSS keyframes	client/style.css	Pet bobbing, blinking, arrow exits and collision feedback; reduced-motion support.
Capacitor 8	android and package.json	Packages the player frontend as an Android WebView application.
FastAPI and Pydantic	backend/app	HTTP routing, input validation and automatic OpenAPI docs.
SQLAlchemy 2	db.py and models.py	Shared relational model for SQLite development and PostgreSQL deployment.
PostgreSQL	Production database	Transactions, uniqueness constraints and account row locks for economy changes.
bcrypt and HMAC	security.py	Password hashes, hashed session tokens and hashed OTP codes.
SMTP and Twilio	providers.py	Sends generated codes to email or phone; live credentials required.
Google OAuth	routes/auth.py	Browser authorization-code flow with an extra email OTP.
Docker and pytest	Root and backend/tests	Repeatable deployment configuration and automated rule/API tests.

The API is authoritative. The client never supplies a balance, final star score, elapsed time or claimed difficulty. It sends moves; the server checks the board, updates state and computes completion. Every arrow occupies one grid cell and moves along a straight ray. This release does not implement long bent multi-cell arrows, multiplayer competition or an offline reward queue.

7 Module and database reference

Module	Responsibility
backend/app/game.py	Pure puzzle generator, collision ray, difficulty tier, score and payout.
backend/app/db.py	Engine, sessions and transaction lifetime; SQLite write serialization.
backend/app/models.py	All database entities and uniqueness constraints.
backend/app/security.py	Password hashing, session validation, contact validation and rate counters.
backend/app/providers.py	Delivery adapters for console development, SMTP and Twilio.
backend/app/routes/auth.py	Registration, sign-in, resets, linked contacts and Google OAuth.
backend/app/routes/play.py	Attempts, moves, lives, claims, streaks, history and heartbeat.
backend/app/routes/profile.py	Username, gender, photo upload and user reviews.
backend/app/routes/admin.py	Protected summary metrics, directory, player records and reviews.
client/src/game.js	Board display, movement, timer, retry dialogs and sound triggers.
client/src/app.js	Home, level selection, journey, navigation and wallet display.
admin/admin.js	Owner dashboard, comparisons, charts, search and player details.

Data entities

Table	Important stored information
users	Numeric string ID, unique contacts and username, password hash, profile, balances, unlocked level, streak dates and role.
attempts	Owner ID, level, board JSON, event JSON, lives, purchases, collisions, state, timestamps, stars and reward decision.
rewards	Owner ID plus unique reward key, coin and diamond deltas and timestamp. Negative deltas record life purchases.
sessions	Hashed bearer token, user ID and expiry.
challenges	Hashed OTP, contact, purpose, pending payload, attempt count, expiry and used flag.
reviews	One current rating and optional text per user.
rate_limits	Hashed rate-limit key, window start and request count.
oauth_flows	Google state, hashed polling secret, expiry, challenge link and completion flag.

Users relate one-to-many to attempts, rewards and sessions, and one-to-one to a current review. docs/schema.sql is a PostgreSQL DDL reference; docs/ARCHITECTURE.md includes the editable entity diagram. User IDs are stored as strings so JavaScript does not accidentally change their digits.

8 Authentication and transaction behavior

Registration lifecycle

The signup request validates the contact and username, hashes the password and creates a pending challenge. It does not create the final user until OTP verification succeeds. Codes use a cryptographic random generator and contain exactly four digits, including possible leading zeroes. The database stores an HMAC digest instead of the plaintext OTP. A five-minute expiry and five failed verification attempts limit each challenge.

Delivery and sessions

SMTP sends email over STARTTLS. Twilio's Messages endpoint sends the server-generated code by SMS. Production must use `OTP_MODE=live`. Console mode prints codes only for local development and is refused by the production configuration check. Successful verification returns a random bearer token. Its digest is stored on the server and expires after seven days. The frontend keeps the token in `sessionStorage`, so closing the app session may require signing in again.

Duplicate requests and balances

Authenticated game actions lock the user row in PostgreSQL. The SQLite development dependency begins an immediate transaction. Reward keys are unique per user, preventing duplicate daily, streak, level and life ledger entries. Repeated moves include a request ID; a retry of the same request returns current state without removing another life. Buying a life requires an exhausted attempt, enough coins and fewer than three previous purchases.

Boundaries to know

Server validation prevents fabricated successful moves and repeated reward claims. It does not prove a human solved the puzzle: automated solvers can inspect the board. There is no competitive leaderboard or real-money economy in this release. Four-digit codes are short, so retain the expiry and attempt limits; protect deployment endpoints with appropriate request throttling if public usage grows.

No user found intentionally identifies an unknown registered contact because the requested recovery flow calls for that message. Before a broad public launch, decide whether to switch to a generic recovery response to reduce account enumeration. Current rate limits are a development-scale database implementation, not a high-volume abuse-control system.

Device time and time zones

The displayed date and time follow the device locale. Durations are server-derived elapsed time, while the visible counter uses a monotonic browser clock between server updates. Streak dates use the registered account time zone and the server clock. The account time zone is fixed after registration in this version; a device clock change cannot mint extra rewards. Google-first accounts default to UTC. Travel-related automatic time-zone changes are not implemented.

9 Run locally on Windows

1. Install Python 3.12, Node.js 22 or newer and VS Code. Extract EscapeArrow_Project.zip into a normal folder, then open that folder in VS Code.

2. Open Terminal and run the commands below from the extracted project. They use the virtual environment executable directly, so PowerShell activation policy changes are unnecessary.

```
cd backend
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
Copy-Item .env.example .env
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
```

1. Open <http://localhost:8000> in a browser. Do not double-click the HTML file; modules and API calls require the server.

2. Register with a test contact. Read the printed DEVELOPMENT OTP from the running backend terminal and enter it in the app. No message is delivered in console mode.

3. Start level 1, clear the arrows, collect the reward and verify level 2 unlocks. Try the profile, review and history screens.

4. Use a second terminal in backend to promote your verified owner account, then open <http://localhost:8000/admin/>.

```
.\.venv\Scripts\python.exe manage.py make-admin your@email.com
```

Run on macOS or Linux

```
cd backend
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
cp .env.example .env
.venv/bin/python -m uvicorn app.main:app --reload --port 8000
```

Files created at runtime

SQLite creates backend/escape.db. Treat it as local data rather than source code. The server creates missing tables on startup. .env contains your settings and must stay private. The API reference is available at /docs and the machine-readable schema at /openapi.json. Stop the local server with Ctrl+C. Use a managed PostgreSQL database for an online installation.

Automated checks

```
cd backend
.\.venv\Scripts\python.exe -m pytest -q
```

Tests use a temporary database and stub OTP delivery. They do not send messages or consume your provider account. Run npm ci and npm run build from the project root to validate the player bundle separately.

10 Connect PostgreSQL and configure providers

Recommended production database

A managed PostgreSQL database, such as Supabase Postgres, fits this project because profiles, attempts and reward ledgers need relational constraints and transactions. The FastAPI server connects using SQLAlchemy and psycopg; the Android app never connects directly to the database. Supabase Auth is not used because this project owns its four-digit OTP workflow. Supabase offers direct and pooled PostgreSQL connections; select the connection type suitable for your server network from its Connect panel [1].

1. Create a PostgreSQL project and record its database password privately. In Supabase open Connect and choose a direct connection or session pooler that your host can reach.
2. Copy the supplied connection information. In the backend environment, set DATABASE_URL using the SQLAlchemy psycopg scheme below. Encode reserved password characters in the URI. Do not paste this URI into client/config.js.
3. Use SSL in the connection string. Start the backend or run `python manage.py init-db`. Tables are created in the database. Inspect them in the provider's table viewer.
4. Register a new account against this backend and complete a level. Confirm users, attempts and rewards rows are present. Backups and credentials belong to the database provider account.

```
DATABASE_URL=postgresql+psycopg://USER:PASSWORD@HOST:5432/postgres?sslmode=require
```

Email and phone codes

Set `OTP_MODE=live`. For email configure `SMTP_HOST`, `SMTP_PORT=587`, `SMTP_USER`, `SMTP_PASSWORD` and `SMTP_FROM` with a provider that supports authenticated STARTTLS. Verify the sender with the provider and test a real inbox. For SMS configure `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN` and `TWILIO_FROM`. The adapter sends a generated code using Twilio Programmable Messaging [2]. Enable the destination regions and sender setup in your provider account; delivery and charges are provider-managed. Provider credentials are not included.

Google configuration

1. Create a Google Cloud OAuth consent screen and a Web application OAuth client for the backend browser flow. Add your test users while the consent screen is in testing mode.
2. Set the authorized redirect URI to `https://YOUR-BACKEND-DOMAIN/api/auth/google/callback` exactly. Set `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` on the backend.
3. Set `PUBLIC_URL` to that same HTTPS domain. Configure live email delivery, because Google sign-in is followed by the required email OTP.
4. Test the full flow: open Google, approve access, return to EscapeArrow, press I have signed in and enter the email code. Android uses a system browser rather than an embedded Google login [3].

11 Deploy the backend and dashboard online

Use a host that can run the supplied Dockerfile as a long-running web service. The player web interface and administrator dashboard are served by the same FastAPI process. A static-only host cannot run this backend. Do not put the database on ephemeral application storage.

1. Create a private Git repository containing the source. Exclude `.env`, `*.db`, `node_modules`, temporary files and signing keys. Commit the package lock and Android source.
2. Create a Docker web service on your chosen host and connect the repository. Set the Docker build context to the project root. The Dockerfile installs Python dependencies and starts Uvicorn on PORT or 8000.
3. Configure `DATABASE_URL` for managed PostgreSQL, `APP_ENV=production`, `OTP_MODE=live`, `PUBLIC_URL=https://YOUR-BACKEND-DOMAIN` and a long random `APP_SECRET`. Add the SMTP, Twilio and Google variables you intend to support.
4. Set `ALLOWED_ORIGINS` to the public web origin and `https://localhost` for the Android WebView. Use comma-separated exact origins. Keep provider secrets only in the host's protected environment settings.
5. Deploy and wait for `/health` to return status ok. Open the player site, register with a real contact, complete verification, play a level and verify database writes.
6. Use the host's terminal with the same environment to run `python manage.py make-admin your@email.com` from `/app/backend`. Open `/admin/` and confirm reporting works.
7. Build the APK against this final HTTPS URL. A test phone must be able to reach it over the network. Keep the backend online for authenticated gameplay.

Generate a server secret

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Store the generated value as `APP_SECRET`. Changing it invalidates token and OTP digests and requires players to sign in again. Keep the same value across backend instances. Production startup intentionally fails if the secret is weak, OTP mode is console, the database is not PostgreSQL or `PUBLIC_URL` is not HTTPS.

Docker on your own machine

The supplied `docker-compose.yml` runs PostgreSQL and the API together. Copy `backend/.env.example` to `backend/.env` first. Set a strong `POSTGRES_PASSWORD` in your terminal, then run `docker compose up --build`. Use a URI-safe password for the sample compose interpolation. For a public server add an HTTPS reverse proxy, backups and a private database network. FastAPI documents the container deployment approach [4].

Data upgrades and backups

Initial tables are created automatically, but `create_all` does not alter existing columns. Before changing models in later releases, add an explicit versioned database migration, back up the database and verify the migration on a copy. Use scheduled provider backups or `pg_dump`, and test restoring into a separate database. Deleting a deployment must not delete the persistent database.

12 Build and install the Android APK

Prepare Android Studio

The source includes the generated Capacitor Android project. Install Node.js 22 or later and Android Studio 2025.2.1 or later. Use Android Studio's bundled JDK and install Android SDK Platform 36, build tools and platform tools. The project targets SDK 36 and sets minimum SDK 24. These requirements follow Capacitor 8 documentation [5].

Connect the app to your server

1. Open a terminal at the project root, not backend. Install locked Node dependencies with `npm ci`.
2. Set `API_URL` to the deployed HTTPS backend. Then build and sync the player assets into the existing Android project. Do not run `android:add` again because `android/` already exists.
3. Open the native project in Android Studio and let Gradle sync finish. If Android Studio asks for the SDK path, select your installed SDK.

```
npm ci
$env:API_URL="https://YOUR-BACKEND-DOMAIN"
npm run android:sync
npm run android:open
```

On macOS/Linux use `API_URL=https://YOUR-BACKEND-DOMAIN npm run android:sync`. You can alternatively edit `client/config.js` before building. With an empty API base the browser uses the current web origin; a packaged Android app needs your remote backend URL. Never use `localhost` as the hosted backend URL for a physical phone.

Build a debug APK

1. In Android Studio select a connected device or emulator and use Run to test the app. Enable developer USB debugging on your own test phone if using a cable.
2. Use Build > Generate App Bundles or APKs > Generate APKs, or the equivalent APK build action in your Android Studio version. A terminal alternative from `android` is `.\gradlew.bat assembleDebug` on Windows or `./gradlew assembleDebug` on Linux/macOS.
3. Find `android/app/build/outputs/apk/debug/app-debug.apk`. Install it on your test device and verify sign-in, gallery upload, sound, collision, restart, rewards and small-screen touch behavior.

Create the release file

Use Build > Generate Signed App Bundle or APK. Choose APK for direct installation or Android App Bundle for store distribution. Create a signing keystore and store it privately with its passwords. Use the same key for updates; increase `versionCode` in `android/app/build.gradle` for each release. Release signing credentials are intentionally not supplied. Review your store's current publishing requirements before submission.

For every frontend edit, rebuild and run `android:sync` before building again. Backend-only edits require a server deployment. The administrator dashboard remains a separate browser interface, so private admin screens are not bundled into the player APK.

13 API reference for accounts and profile

All paths below are relative to the backend origin. JSON requests use Content-Type: application/json. Protected endpoints require Authorization: Bearer TOKEN. Never send tokens in URLs. See docs/openapi.json or the running /docs page for every field and validation rule.

Method and path	Input	Result
POST /api/auth/start	purpose, contact; signup adds name, username, password, timezone	challenge_id; sends code
POST /api/auth/verify	challenge_id, four-digit code; reset adds password	Session token and profile
POST /api/auth/logout	Authenticated session	Revokes the current token
POST /api/auth/google/start	Empty object	Browser URL, state and polling key
GET /api/auth/google/callback	Google code and state	Exchanges code; sends email OTP
POST /api/auth/google/poll	state and polling key	challenge_id when ready
POST /api/auth/contact/start	Authenticated; new contact	Linked-contact challenge
POST /api/auth/contact/verify	Authenticated; challenge_id and code	Verifies and stores contact
GET /api/me	Authenticated	Profile, balance, daily availability
PATCH /api/profile	username and gender	Updated profile
POST /api/profile/avatar	Multipart file field named file	Updated profile with JPEG data
POST /api/reviews	stars 1-5, optional text up to 2000 characters	Saves current player review
POST /api/heartbeat	Authenticated	Updates online timestamp

Example account request

```
POST /api/auth/start
{
  "purpose": "signup",
  "contact": "player@example.com",
  "name": "Player Name",
  "username": "arrow_player",
  "password": "UseYourOwnStrongPassword",
  "timezone": "Asia/Kolkata"
}
```

purpose also accepts login, otp and reset. login requires the password. The API returns a challenge ID rather than the OTP. For local tests the code is printed only in the server terminal. Never put a provider password in an API request from the frontend.

14 API reference for game and reporting

Method and path	Purpose
GET /api/levels?offset=0	Thirty level cards with unlock and best-result information.
POST /api/attempts	Start a level; body contains level. Ends any older active attempt.
GET /api/attempts/active	Retrieve the current playing or exhausted attempt.
POST /api/attempts/{id}/move	Body contains arrow_id and unique request_id. Server validates the ray.
POST /api/attempts/{id}/life	Spend the next life price after reaching zero lives.
POST /api/attempts/{id}/reward	Body contains collect true or false. Decision is final per attempt.
POST /api/attempts/{id}/exit	End an active attempt as abandoned.
POST /api/daily/claim	Claim two coins after starting a level on the current date.
GET /api/history	Latest 100 attempt records for this account.
GET /api/wallet	Latest 100 reward or life-purchase ledger entries.
GET /api/admin/overview	Protected aggregate counts, time averages, charts and tier metrics.
GET /api/admin/players	Protected directory; q filter and offset pagination, 50 rows.
GET /api/admin/players/{id}	Protected profile and latest 200 attempts.
GET /api/admin/reviews	Protected reviews; offset pagination, 50 rows.
GET /health	Deployment readiness response.
GET /openapi.json	Live machine-readable API schema.

Example move

```
POST /api/attempts/ATTEMPT_ID/move
Authorization: Bearer YOUR_SESSION_TOKEN
{ "arrow_id": 4, "request_id": "a-new-unique-request-id" }
```

The response includes the authoritative board, lives, collisions, purchases, status, timer fields, score and payout. Reuse the same request_id only when retrying that exact move. A new interaction must have a new ID. Reward collection and extra-life spending are separate endpoints so the client cannot directly alter balances.

Common HTTP responses

400 means an invalid code or game action. 401 means authentication is missing or expired. 403 means a locked level or insufficient role. 404 means no matching account or owned attempt. 409 means a conflicting username, contact or attempt state. 429 means the request limit is reached. 503 means an external account provider is unavailable or unconfigured.

15 Testing troubleshooting and references

What has been verified

Nine API and rule tests pass. They cover 180 generated boards, completing and unlocking levels, duplicate rewards and moves, declining rewards, low scores, five collisions, life prices and limits, daily and streak grants, missing-day reset, access control, OTP replay and limits, password-reset session revocation and linked-contact verification. The frontend bundle builds and Capacitor generation and synchronization succeed. See docs/VALIDATION.md for the final validation record.

What to test before distributing

Configure and test live SMTP, SMS and Google OAuth. Run the integration suite against a test PostgreSQL database and exercise concurrent reward requests. Build and run the APK on physical Android devices. Confirm portrait layouts, file selection, sound, slow network behavior and session recovery. Test production backup restoration. This release does not claim these external checks have already passed.

Symptom	What to check
OTP not arriving	Console mode prints it in the backend terminal. Live mode requires working credentials, verified senders and permitted recipients.
Google button unavailable	Set GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET and deploy the exact authorized callback URL.
Failed to fetch in APK	Check API_URL, HTTPS reachability and ALLOWED_ORIGINS including https://localhost; rebuild and sync assets.
No user found	Use the exact registered contact. Phone numbers include the country code. Verify a second contact from Profile before using it.
Level remains locked	The preceding level must be completed with at least 1.5 stars. A completion below the threshold still requires a replay.
Reward does not increase balance	A reward can be collected once per attempt. No thanks is final. Check Wallet activity and server logs.
Database connection fails	Check the host, encoded password, SSL and direct versus session-pooler network compatibility.
Android Gradle fails	Install SDK 36 and use Android Studio's supported JDK. Run npm ci and android:sync again after fixing dependencies.

Official implementation references

- [1] Supabase PostgreSQL connections: <https://supabase.com/docs/guides/database/connecting-to-postgres>
- [2] Twilio Messages API: <https://www.twilio.com/docs/messaging/api/message-resource>
- [3] Google OAuth web server flow: <https://developers.google.com/identity/protocols/oauth2/web-server>
- [4] FastAPI Docker deployment: <https://fastapi.tiangolo.com/deployment/docker/>
- [5] Capacitor environment setup: <https://capacitorjs.com/docs/getting-started/environment-setup>
- [6] Capacitor Android: <https://capacitorjs.com/docs/android>